

Data Analysis for AI/ML

Complete Revision Notes

AI/ML Engineering Revision Journey — Module 3 (Day 1 & Day 2)

Covers: What Data Analysis is · Types of Data · Data Collection · Data Quality & Cleaning · Exploratory Data Analysis (EDA) · Data Visualization · Feature Engineering · Data Preprocessing · Data Splitting · Essential Python Libraries

Introduction

In Artificial Intelligence and Machine Learning, data is the raw material from which intelligence is created. A model does not understand the real world directly — it learns patterns, relationships, and behaviors from the data provided to it.

A powerful algorithm with poor-quality data can fail, while a well-prepared dataset can significantly improve model performance.

This makes Data Analysis a critical foundation of every successful AI system. These notes cover the full journey from understanding raw data to preparing it for a Machine Learning model.

1. What is Data Analysis?

Data Analysis is the process of inspecting, cleaning, transforming, and interpreting data to discover useful information, patterns, and insights. It helps answer some important questions:

- What does the data tell us?
- Are there hidden patterns?
- Which factors influence outcomes?
- Is the data ready for machine learning?

2. Why Data Analysis is Important for AI/ML

Before training any ML model, data analysis helps us:

- Understand the structure and behavior of datasets
- Detect errors and inconsistencies
- Identify important features
- Improve data quality
- Make better decisions during model development

The accuracy of an AI model depends not only on the algorithm but also on how well we understand and prepare the data.

3. Types of Data

Type	Description
Qualitative Data	Descriptive information such as categories and labels.

Quantitative Data	Numerical information used for mathematical analysis.
Structured Data	Organized data stored in tables and databases.
Unstructured Data	Text, images, audio, and other complex formats.

4. Data Collection

Data can be collected from multiple sources, including:

- Databases (SQL / NoSQL)
- APIs
- Web Scraping
- Surveys
- Sensors & IoT Devices

The quality and relevance of collected data directly affects the quality of the resulting AI solution.

```
# Common ways to load real-world data
df = pd.read_csv("students.csv")
df = pd.read_json("students.json")
df = pd.read_sql("SELECT * FROM students", connection)
```

5. Data Quality & Data Cleaning

Real-world datasets are rarely perfect. Common issues include:

- Missing Values
- Duplicate Records
- Incorrect Data
- Inconsistent Formats
- Outliers

Data Analysis Workflow

Collect Data → Clean Data → Explore Data → Analyze Patterns → Extract Insights → Prepare Data for ML Models.

Code: Detecting issues

```
# Missing values
df.isnull().sum()

# Duplicate rows
```

```
df.duplicated().sum()
```

```
# Outliers using the IQR method
```

```
Q1 = df["study_hours"].quantile(0.25)
```

```
Q3 = df["study_hours"].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
lower, upper = Q1 - 1.5 * IQR, Q3 + 1.5 * IQR
```

```
outliers = df[(df["study_hours"] < lower) | (df["study_hours"] > upper)]
```

6. Exploratory Data Analysis (EDA)

EDA means exploring a dataset before model development. It typically covers:

- Understanding dataset structure
- Summary statistics
- Identifying data distributions
- Finding relationships between variables
- Detecting missing values and outliers

EDA helps answer an important question: "What story is the data trying to tell?"

```
df.info()
df.describe()
df["study_method"].value_counts()
df.corr(numeric_only=True) # correlation matrix
```

7. Data Visualization

Visualizations simplify complex datasets and reveal trends, patterns, and anomalies that are difficult to identify from raw numbers alone. The most commonly used chart types:

Chart	Best used for
Histogram	Distribution / spread of a single numerical variable
Bar Chart	Comparing counts or values across categories
Line Chart	Trends over an ordered sequence (e.g. time)
Scatter Plot	Relationship between two numerical variables
Box Plot	Spread, median, and outliers across groups
Heatmap	Correlation strength between multiple variables

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.histplot(df["score"], bins=10, kde=True) # Histogram
sns.countplot(data=df, x="study_method") # Bar Chart
plt.plot(df["student_id"], df["score"]) # Line Chart
sns.scatterplot(data=df, x="study_hours", y="score") # Scatter Plot
sns.boxplot(data=df, x="study_method", y="score") # Box Plot
sns.heatmap(df.corr(numeric_only=True), annot=True) # Heatmap
```

8. Feature Engineering

Well-designed features often contribute more to model performance than simply choosing a more complex algorithm.

- **Feature Selection** — keeping only relevant columns
- **Feature Creation** — deriving new, more meaningful columns
- **Removing Irrelevant Features** — dropping columns with no predictive value
- **Encoding Categorical Variables** — converting text categories into numbers

```
# Feature creation
df["study_efficiency"] = df["score"] / df["study_hours"]

# Removing an irrelevant feature
df = df.drop(columns=["student_id"])

# Label Encoding
from sklearn.preprocessing import LabelEncoder
df["study_method_label"] = LabelEncoder().fit_transform(df["study_method"])

# One-Hot Encoding
df_encoded = pd.get_dummies(df, columns=["study_method"])
```

9. Data Preprocessing

Proper preprocessing ensures that Machine Learning algorithms receive clean and consistent input data.

- Handling Missing Values
- Encoding Categorical Data
- Feature Scaling
- Normalization
- Standardization

Normalization vs. Standardization

Technique	What it does	Typical range
Normalization (Min-Max)	Rescales values into a fixed range	[0, 1]
Standardization (Z-score)	Rescales values to mean 0, std 1	Centered around 0

```
from sklearn.preprocessing import MinMaxScaler, StandardScaler

# Handling missing values
df["study_hours"] = df["study_hours"].fillna(df["study_hours"].median())

# Normalization -> range [0, 1]
df[cols] = MinMaxScaler().fit_transform(df[cols])

# Standardization -> mean 0, std 1
df[cols] = StandardScaler().fit_transform(df[cols])
```

10. Data Splitting

Before training any model, datasets are divided into:

- **Training Set** — used to fit the model
- **Validation Set** — used to tune the model and check generalization during development
- **Test Set** — used only at the end, to evaluate real-world performance

This allows us to evaluate how well a model generalizes to unseen data rather than simply memorizing the training dataset.

```
from sklearn.model_selection import train_test_split

X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.30, random_state=42)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.50, random_state=42)
```

11. Essential Python Libraries

Library	Purpose
NumPy	Numerical computing, arrays, math operations
Pandas	Loading, cleaning, and manipulating tabular data
Matplotlib	Base plotting library for visualizations
Seaborn	Statistical visualizations built on Matplotlib
Scikit-learn	Preprocessing, encoding, scaling, splitting, ML models

12. Full Data Preparation Pipeline

Putting every concept together, a typical data preparation pipeline looks like this:

```

Raw Data
-> Data Cleaning      (missing values, duplicates, outliers)
-> EDA                (summary stats, distributions, correlations)
-> Feature Engineering (new features, selection, encoding)
-> Encoding & Scaling (label/one-hot encoding, normalization/standardization)
-> Train / Validation / Test Split
-> Machine Learning Model

```

Key Takeaways

- Data Analysis is the foundation every AI/ML model is built on — garbage in, garbage out.
- EDA answers "what does the data look like?" before any model is trained.
- Visualizations reveal patterns and anomalies that raw numbers hide.
- Feature Engineering often improves performance more than switching algorithms.
- Preprocessing (cleaning, encoding, scaling) makes data consistent and model-ready.
- Splitting data properly lets us measure real generalization, not memorization.

Next up (Module 4): Machine Learning — Regression, Classification, Clustering, Model Evaluation, Scikit-learn.